# —--Iterable interface—--

**Iterable interface**→It is the root interface that represents any object whose element can be traversed one by one.

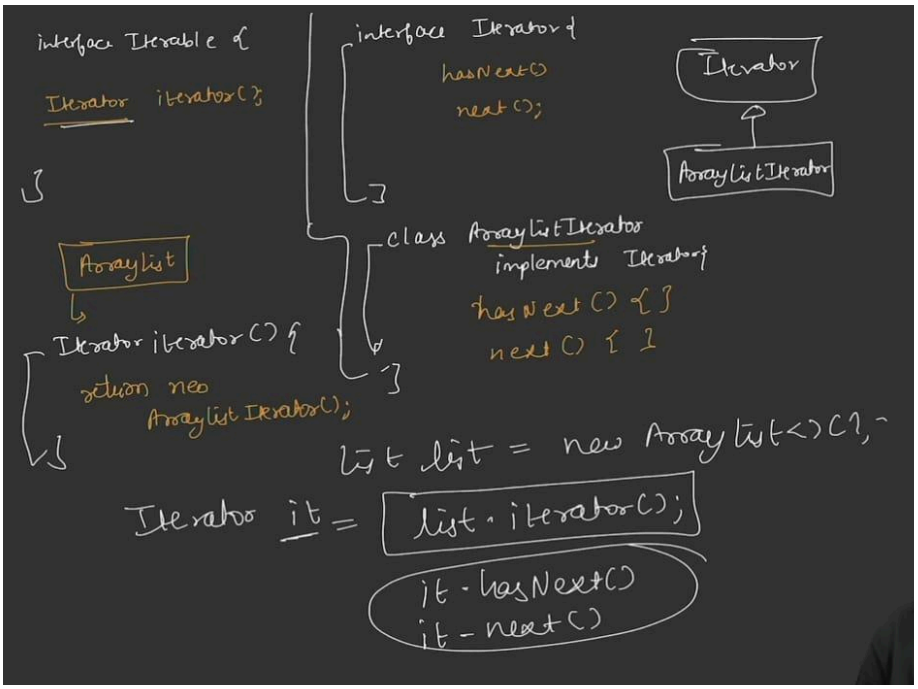
---

**There are 2 types of DataStructures:-**

1) **Ordering maintaining DataStructures.**
- Only Dynamic ArrayList is index based.
    - ArrayList
    - LinkList
2) **Not maintaining Ordering DataStructures.**
    - Set
    - Map
    - Stack
    - Queues

```java
import java.util.*;
public class IterablesIntro {
    public static void main(String[] args) {
        Collection <Integer> list = new HashSet<>();
            list.add(10);
            list.add(20);
            list.add(30);
            list.add(40);
            list.add(50);
                Iterator <Integer> it = list.iterator();
                    while(it.hasNext()){
                        System.out.println(it.next());
                    }
    }
}
// 'list.iterator();' this method is in List interface.
// Iterator<Integer> ka use isliy kr re ha taki 'hasNext()', 'next()'. mil sake.
```

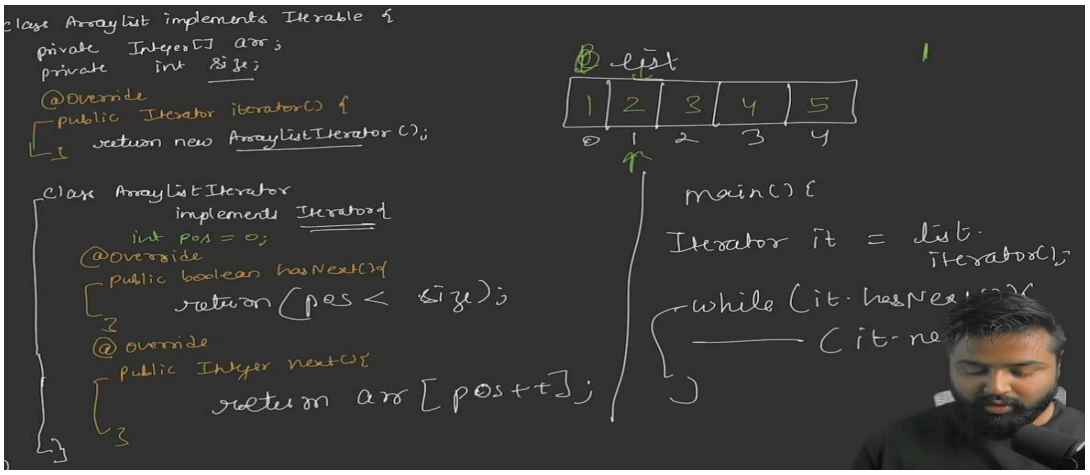
Internally →



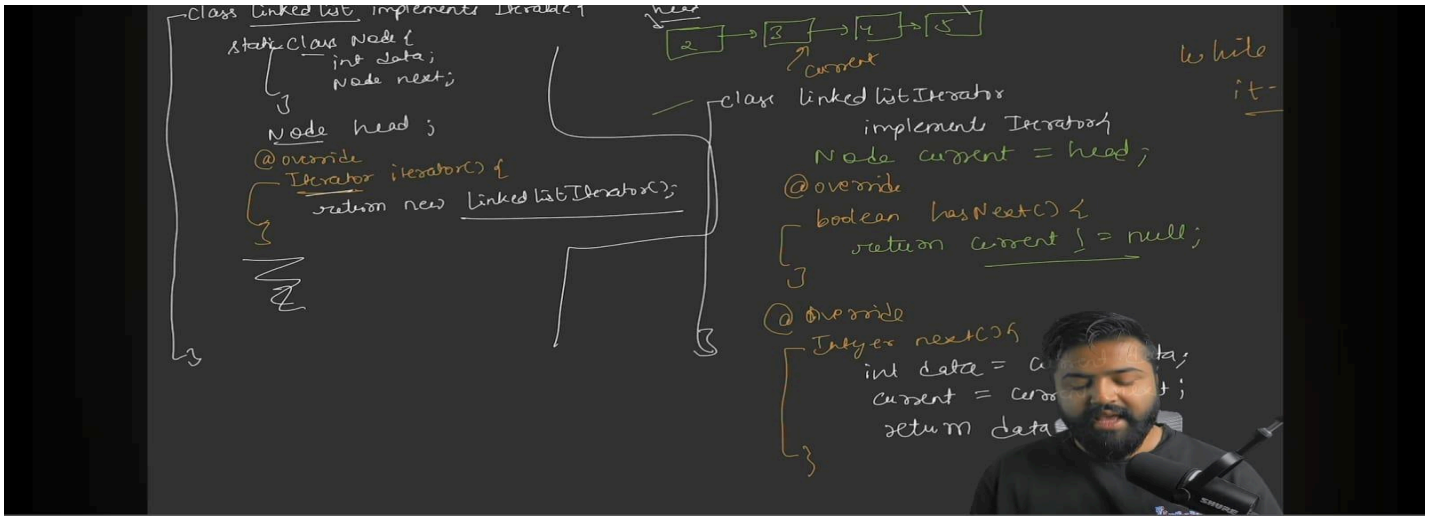

---

## How Different **Collection** conceptually implements **ITERABLE / ITERATOR**

| **ArrayList:** |
|---|




**LinkedList:**




My own Iterable Interface

```java
import java.util.Iterator;

public class OwnIterator {
    public static void main(String[] args) {
        String[] names = {"Aditya, Rahul, Rohit, Saurav, Priyansh"};
        NameContainer container = new NameContainer (names);
            Iterator<String> it = container.iterator();
                while(it.hasNext()){
                    System.out.println(it.next());
                }
    }
}
class NameContainer implements Iterable<String>{
    private String[] names;
    private int size;
        NameContainer(String[] names){
            this.names = names;
            this.size = this.names.length;
        }
    @Override
    public Iterator<String> iterator(){
        return new NameContainerIterator();
    }
        private class NameContainerIterator implements Iterator<String>{
            private int position = 0;
                @Override
                public boolean hasNext(){
                    return position < size;
                }
                @Override
```

```
            public String next(){
                return names[position++];
            }
        }
}
```

More concepts of Iterator→

